A Lock-Free Data Plane and the Signal-to-Seed Bottleneck in Nanopore Adaptive Sampling

A briefing for a reader outside the field

Khushal Singh

This document explains what the paper is about, what it claims, and how each claim can be checked. It assumes familiarity with real-time systems, concurrency and hashing, and assumes nothing at all about DNA sequencing. Every domain term is defined where it first appears, and there is a glossary at the end. Section numbers in the margin notes refer to the paper.

---

1 The instrument, in one page

A *nanopore sequencer* reads DNA by pulling a single molecule through a hole roughly one nanometre wide. An ionic current flows through that hole, and the molecule occupying it changes how much current gets through. Different sequences of bases obstruct the pore differently, so the current trace over time is a noisy, indirect record of the sequence.

Three consequences matter for this paper.

The instrument emits signal, not letters. What comes off the device is a stream of current measurements at 4 kHz, not a string over `{A,C,G,T}`. Converting one to the other is called *basecalling* and is done by a neural network. It is accurate and it is expensive.

A device has many pores at once. A production flow cell runs 512 pores, called *channels*, in parallel and independently. Each is reading a different molecule.

Reading is destructive of time, not of the molecule. A molecule takes seconds to transit the pore. While it is transiting, the channel is occupied and cannot read anything else.

That last point is what creates the opportunity. The manufacturer exposes an interface called **Read Until**: a client program receives the current trace in chunks as it is produced, and may at any point tell the device to *eject* the molecule in a given channel. The channel is then free for the next one. If the client can tell, early, that a molecule is not one the experiment cares about, ejecting it buys back the seconds the rest of its transit would have consumed. The yield of wanted DNA goes up, with no change to the laboratory protocol. This is *adaptive sampling*.

2 Why this is a real-time problem, and an unusual one

A chunk of signal describes bases that have *already* passed through the pore. So a decision that arrives after the molecule has finished transiting is not a late decision; it is a decision about a molecule that has already gone. Its value is zero, not reduced.

That gives a hard budget. With C channels each delivering a chunk every T_c seconds, and a tolerance of W chunks before a decision is worthless, the mean time available per decision is

$$S < \frac{W \cdot T_c}{C},$$

which at the production values $W = 2$, $C = 512$, $T_c = 0.4\,\text{s}$ gives **1.56 ms**.

Two framings follow, and they are the intellectual core of the first half of the paper.

Soft real time. Utility falls to zero at the deadline rather than degrading. So the quantity of interest is the *distribution* of decision latency, not its mean.

A loss system, not a delay system. Under overload, a client must choose between queueing work and discarding it. Queueing is the wrong choice here: a queued decision becomes a decision about a molecule that has already left. Discarding is correct, not merely pragmatic.

The consequence is the observation the paper is organised around, and it is worth stating plainly to anyone assessing work of this kind:

> **A mean comfortably inside the budget is consistent with an arbitrary deadline-miss rate.** A system reporting "mean 124 µs against a 1.56 ms budget" has said nothing about whether it meets its deadline.

This is why the paper reports nine runs with full ranges and names the one that crosses, rather than a single comfortable average. It is a weaker-sounding claim and a more useful one.

3 What the paper claims

Two contributions, and they point in opposite directions. The first is positive and the second is negative, and the author argues the negative one is the more valuable.

Contribution 1: the decision budget is not the obstacle (§III–§IV)

A single-process C++20 client was built: a *data plane* that ingests signal, decides, and emits eject commands, without allocating memory or taking a lock on the decision path.

- Median per-chunk decision latency **124.8 µs** against the 1.56 ms budget — about 8% utilisation — at genome scale, over a 64 Mb human chromosome 20 reference, with no chunks dropped across nine runs.
- Peak memory **3.70 GiB**, including the transient cost of building the index.
- For calibration, an existing basecalling client (*readfish*) was run with its neural network replaced by a no-op, so that it does no basecalling and no mapping at all. Its remaining orchestration alone costs **124.3 µs** per chunk.

That last figure is the point of the comparison. The bookkeeping of a standard client, *before any decision work whatsoever*, costs as much as this system's complete decision. So microsecond-scale decisions are achievable, and the deadline is not what stands between this approach and deployment. Hence the paper's original title, which has since been shortened.

Contribution 2: the real limit is before the decision (§V–§VII)

Here is the structure of the matching problem. The reference genome is converted into the signal one would *expect* it to produce. The observed signal is cut into *events* (one per base-ish step), each event is *quantised* to a small integer symbol, and E consecutive symbols are packed into a *key* that is looked up in a hash table. A hit is an *anchor*: evidence that this stretch of signal came from that place in the genome. Enough consistent anchors, and the molecule is identified.

The pipeline is strictly ordered:

$$\text{reference encoding} \rightarrow \text{segmentation} \rightarrow \text{quantisation} \rightarrow \text{seeding} \rightarrow \text{chaining}$$

and **no later stage can recover evidence an earlier stage destroyed**. This is the spine of the whole second half.

The headline comparison: on identical input, this pipeline resolves **17.6%** of reads where the published state of the art, RawHash2, resolves **91.8%**. A factor of about five. The paper's job from there is to explain that gap rather than excuse it — and crucially, to show that it is *not* caused by the decision rule, which is the part most easily blamed and most easily tuned.

4 The technical heart: why a key matches, or does not

Let a be the probability that the reference and the query assign the *same* quantiser symbol to corresponding events. A key packs E events and matches only if all E agree, so anchor yield falls as a^E.

The author originally argued a^E was a *lower* bound, with a plausible argument: consecutive events overlap, and per-read normalisation errors move all of a read's values together, so agreements cluster rather than being independent.

That argument was then refuted by the author's own measurement. It holds only if the two event sequences correspond one-to-one. If one side emits an event the other does not, the key window is misaligned and cannot match however well its neighbours agree. Writing g for the rate at which the two sequences fail to correspond at all, the governing form is

$$P(\text{key}) \approx (1-g)^E \cdot a^E,$$

and the first factor is simply absent from the original. Measured, a^E turns out to *over*-estimate by up to 14× wherever segmentation is data-dependent.

Self-refutation is unusual in a thesis and it is deliberate. The earlier claim is stated, the condition under which it holds is identified, and the measurement that breaks it is given.

The measurement instrument, and why it is fair

An agreement figure computed one way for your system and another way for the competitor is not a comparison. So both systems were made to emit their per-event symbols in one common format, and a single scoring program — which *does not know which system produced its input* — aligns the two streams and reports a, g, and the key-match probability measured directly rather than modelled. This required a four-line patch to RawHash2 to expose symbols it dumps internally but does not print.

The estimator symmetry is the methodological contribution of this section, and it is the thing to look for when assessing whether the comparison is honest.

What the measurement found **(§VI–§VII)**

system	a	g	E	$P(\text{key})$	anchors/read
RawHash2	0.911	0.092	8	0.254	467
this work	0.783	0.196	12	0.0055	14.9

A **31-fold** difference in anchor supply — 467 against 14.9 per read — accounts for 91.8% against 17.6% without any appeal to the decision rule at all. The decision stage was never the problem.

Three measured causes, each a design lesson:

1. **Asymmetric event retention.** RawHash2 applies its event-merging rule *symmetrically* to both the reference and the query, keeping 73.4% on each side, so the two sides agree on event counts by construction. This pipeline does not: the reference encoder emits one event per step unconditionally, the query detector about 0.78. The result is an indel rate of **0.170 at zero**

added noise — a structural mismatch between two encoders, not a property of the signal. This is the first thing to fix and it is an engineering change, not a research question.

2. **A quantiser that hoards alphabet.** RawHash2's realised symbol entropy is about **2.46 bits per event**; a third independent system, Sigmoni, lands at 2.58 bits for stated noise-robustness reasons. This pipeline realises more entropy and agrees less. The design axis is per-event entropy against per-event agreement, and spending alphabet to buy agreement is the correct trade.

3. **Recoverable loss in the chainer.** About 11 of 15 anchor-eligible positions per read never reach the best chain. Smaller than the other two, but it is a diagnostic rather than a search, because the anchors are known to exist.

5 Why a negative result is the right outcome here

The paper reports that its own system is roughly five times less accurate than published work. A reasonable question is whether that is publishable. The argument is:

- The cause is *measured*, not speculated: two parameters, a and g, measured under one estimator on identical input for both systems.
- The cause is *modelled*: $(1-g)^E a^E$ predicts the gap from those parameters.
- The model *explains the failures of six attempts to fix it elsewhere.* Six decision rules were tried in triplicate; the best moved accuracy by 35% relative against a gap of a factor of five. The model says why that ceiling exists.
- It yields a prescription a later designer can act on: measure a and g before proposing a decision rule, because together they establish whether *any* decision rule can succeed before one is designed.

There is also a methodological lesson stated plainly in §VIII: **run the external baseline first, not last**. Every improvement to the decision stage was measured against the project's own earlier numbers, which rose and fell credibly and gave no sign that the ceiling lay elsewhere. One run of a published tool on identical input would have established within a day that the gap was a factor of five rather than a tuning problem.

6 What the paper does not claim (§IX)

Worth knowing before reading, because the paper is explicit about all of it:

- **No competitive accuracy.** 17.6% against 91.8% is reported and explained.
- **No novel matching method.** Quantised-event hashing is established work.
- **No trustworthy tail latency.** Every measurement was taken under a hypervisor (WSL2), which can deschedule a virtual CPU mid-decision in a way indistinguishable from a slow decision. The paper treats the median and 90th percentile as properties of the implementation and everything above as contaminated upper bounds. The evidence for that reading is given: maximum latency across nine runs spans 2.0 ms to 59.4 ms, and the run with the largest maximum is *not* the run with the worst 99th percentile. Max and p99 being uncorrelated is the signature of externally injected stalls.
- **All signal is simulated.** Two simulators, whose detector calibrations do not transfer to each other by a factor of thirty. No physical device was used.

- **One organism.** Everything derives from human chromosome 20.

Five further claims could not be settled with the hardware available and are named explicitly as scope boundaries rather than left unmentioned.

7 How every number can be checked

This is the part most relevant to supervision, and it is unusual enough to be worth describing.

The repository contains a script, `verify_paper_numbers.py`, which works in the opposite direction from ordinary checking. Rather than a human transcribing claims and confirming them, the script *rebuilds the expected string from the raw measurement files* under `results/` and asserts that string appears in the manuscript. If a measurement is corrected, the check fails until the paper is updated. It currently reports **144 assertions matched, zero problems**, across all three formats of the manuscript, so no two of them can disagree with each other or with the data.

A second script, `verify_thesis_numbers.py`, does the same for the thesis: 143 claims agreeing, none disagreeing, five documented as outside what the available hardware could settle. A third regenerates every figure from the data and nothing else.

The practical consequence: **any number in this paper can be traced to a file, and the build fails if it cannot be.** Six errors were caught this way during writing, and four of those six were not wrong measurements but *unstated conditions* — which seed, which reference, which decision rule, which window.

8 Questions worth asking

Offered as a starting point for discussion, with where each is addressed:

1. The tail latency is measured under a hypervisor. What would it take to make the deadline-miss claim properly? (§IX, first paragraph)
2. The accuracy comparison is at different operating points — ours at zero false positives, RawHash2 at its own default, which maps 8.4–10.6% of control reads. Is the comparison fair? (§V-A; the paper states it is of magnitude, not of level)
3. If asymmetric retention is the largest cause, why was the decisive ablation not run? (§IX; it is named as not done)
4. The agreement figures rest on two dwell levels and one seed, where accuracy is swept at six levels in triplicate. Is that enough to support dwell-invariance? (§IX, stated as resting on two points)
5. All signal is simulated. What would change on a real device? (§IX)

Glossary

base
: One of the four DNA letters A, C, G, T.

basecalling
: Converting the raw current trace into a sequence of bases, using a neural network. Accurate and expensive.

channel
: One pore on the device, reading one molecule. A production flow cell has 512.

chunk
: One fixed-cadence delivery of signal from one channel.

read
: The signal produced by one molecule's transit through one pore.

event
: One segment of the current trace, corresponding roughly to one step of the molecule through the pore. Produced by *segmentation*.

k-mer / 9-mer
: A window of k consecutive bases. The current depends on about nine bases occupying the pore at once, hence 9-mer.

pore model
: A table giving the expected current for each k-mer. Used to convert a reference genome into the signal it would produce.

quantisation
: Mapping a continuous event level to a small integer symbol, so that events can be hashed and compared exactly.

key E consecutive quantised symbols, packed into one hashable value.

anchor
: A key match between query and reference: evidence of a location.

chaining
: Assembling individually weak anchors into a consistent run that implies a genomic location.

indel
: An insertion or deletion — here, a position where the two event sequences fail to correspond at all, counted by g.

dwell
: How long the molecule rests at each step. Variable in reality; its dispersion is the main source of segmentation noise.

Read Until
: The manufacturer's interface for ejecting a molecule mid-transit.

adaptive sampling
: Using Read Until to raise on-target yield.

data plane
: The part of the client that carries every chunk and makes every decision, as opposed to configuration and setup.

loss system
: A queueing system in which work that cannot be served promptly is discarded rather than queued.